

PROJECT REPORT

SIMShield — Mobile Banking Fraud Protection

Stopping SIM Swaps, eSIM Theft, and Account Takeovers Before Money Leaves the Bank

WHAT WE BUILT	PLATFORMS	MAIN THREAT	TESTS
Full-Stack Security App	Android + Node.js Backend	SIM Swap / Account Takeover	26 out of 26 Passed

1 Problem Understanding

Almost every banking app today uses your phone number as a security key. Need to log in? You get an SMS OTP. Forgot your password? A reset link goes to your phone. Want to add a new payee? Another OTP. Your phone number has basically become the master key to your bank account.

Here's the problem — phone numbers were never designed for this. A scammer with a fake ID can walk into any telecom store, tell them "I lost my SIM card," and walk out with a brand-new SIM card linked to your number. From that moment, every OTP your bank sends goes straight to the scammer's phone, not yours.

This is called a **SIM swap attack**, and it's been growing fast. With eSIMs making things even easier (attackers can hijack your digital SIM profile remotely without even visiting a store), the whole attack can happen in minutes. Here's what a typical attack looks like:

1. The attacker gets control of your phone number (SIM swap or eSIM theft).
2. They request a password reset on your banking app — the OTP goes to them.
3. They log in from a new phone and add their own bank account as a beneficiary.
4. They initiate a large wire transfer. The confirmation OTP? Goes to them again.
5. By the time you notice your phone has "no signal," the money is already gone.

The entire chain — from SIM swap to empty bank account — can happen in under 15 minutes.

Why we can't just block every SIM change: Regular people swap SIMs all the time. You upgrade your phone, you switch carriers, you get a replacement after losing your old one. Blocking accounts on every SIM change would lock out thousands of honest customers for every one scammer caught. We need something smarter.

There's another catch that most people don't think about: **you can't trust the phone itself to detect fraud**. If a hacker has a rooted Android phone with your SIM, they can make the banking app "see" whatever they want — fake device IDs, fake locations, even fake SIM data. Any fraud check that runs inside the phone app can be fooled. The real detection has to happen on the bank's server, where the attacker can't tamper with anything.

2 Proposed Solution

We built **SIMShield** — a system that sits between the telecom carrier and the bank, watching for suspicious patterns in real time. Instead of looking at one event in isolation ("a SIM card changed — panic!"), it watches what happens *after* the SIM changes. If a SIM swap is followed by a password reset from a new device, and then a new beneficiary is added — that's not a coincidence, that's an attack.

How it actually works, step by step:

1. **The phone app doesn't make any security decisions.** It just shows what the server tells it. We call this the "zero-trust client" approach — the phone is treated as an untrusted display screen, nothing more.
2. **Telecom carriers notify our server directly** when a SIM or eSIM event happens. These notifications are signed with cryptographic keys so nobody can fake them.
3. **Our risk engine looks at the last 24 hours of activity.** Each suspicious event adds points to a running score. A SIM swap alone? That's 30 points — you get a gentle "heads up" warning but nothing gets blocked. But a SIM swap plus a new device login plus a password reset? Now you're at 65 points, and we start asking for fingerprint verification before anything sensitive happens.
4. **If the score hits 80 or above, wire transfers get frozen automatically** and a fraud case opens for the bank's security team to review.
5. **Real fingerprint scanning replaces SMS OTP.** When we need to verify someone's identity, we ask for their actual physical fingerprint through the phone's built-in sensor — not an SMS code that the attacker would receive.

6. **A panic button lets users freeze their account instantly.** One tap on "Secure my account" kicks out all other sessions, blocks transfers, and disables SMS-based logins.

The key insight: A SIM change alone is harmless. What makes it dangerous is what happens next. SIMShield watches the sequence of events over time — that's how it catches real attacks without bothering real customers.

3 System Architecture

The system has three main parts, each doing a specific job:



Why this split matters: The backend server is the brain — it's the only thing that decides whether something is suspicious. The phone app is just a screen and a fingerprint reader. The web dashboard is for the bank's security team to monitor everything and take action when needed.

4 Key Features

A. Smart Risk Scoring (Not Just Yes/No)

Every suspicious action adds points to a running total, capped at 100. Here's how much each event is worth:

What Happened	Points	Why It Matters
SIM card swapped or eSIM moved	+30	Someone might have taken over the phone number
Phone number ported to another carrier	+25	Could be a forced number transfer
Login from a phone we've never seen before	+20	Might be the attacker's device
Large or unusual money transfer attempted	+20	Could be the attacker draining funds
Password or PIN reset requested	+15	Attacker trying to lock out the real user
New bank account added as transfer target	+15	Attacker setting up where to send the money
Login from unusual location or time	+10	Could indicate the account is being used elsewhere
Multiple wrong password attempts	+10	Someone guessing the password

B. What Happens at Each Score Level

The response scales with the threat:

- 0 to 29 — LOW Everything's fine. The user doesn't see any warnings or feel any friction at all.

- **30 to 49 — MEDIUM** A heads-up banner appears. "Hey, we noticed a SIM change." The account stays fully open — we don't want to annoy someone who just upgraded their phone.
- **50 to 79 — HIGH** Now we're concerned. Before any sensitive action (transferring money, changing the password), we ask the user to scan their fingerprint. If it's really them, they clear the warning in seconds.
- **80 to 100 — CRITICAL** This looks like a real attack. Outgoing transfers are put on hold immediately, and a case is opened for the bank's security team to investigate.

C. Real Fingerprint Verification

When the app needs to confirm someone's identity, it uses the phone's actual hardware fingerprint reader (Android's built-in `BiometricPrompt` API). This is way more secure than SMS OTP — the attacker would need your physical finger, not just your phone number. If a device doesn't have a fingerprint reader, it falls back to the phone's lock screen PIN or pattern.

D. Emergency Account Lockdown

If a user notices something wrong, tapping "Secure my account" immediately locks things down: all other sessions get terminated, wire transfers are frozen, and new beneficiary additions are blocked. It's like a fire alarm for your bank account.

E. Built-In Attack Simulator

The app comes with 8 simulation buttons so anyone can test what happens during a SIM swap or a full account takeover. There's also a web-based Security Operations Dashboard where the bank's team can watch attacks unfold in real time, review flagged transactions, and manage fraud cases.

5 Technology Stack

Part of the System	What We Used	Why We Picked It
Backend Server	Node.js 20+ (ES Modules)	Fast, lightweight, and handles many events at once without slowing down. We used zero external packages for the core — just Node's built-in HTTP and crypto modules.
Database	PostgreSQL 16	Battle-tested database that keeps our event logs, risk scores, and fraud cases safe with proper transactions. We also have an in-memory option for local testing.
Mobile App	Android (Java, API 26–35)	Covers Android 8.0 through Android 15. Uses the phone's native fingerprint scanner and notification system — no heavy third-party libraries needed.
Hashing & Signatures	HMAC-SHA256	We never store raw phone numbers or device IDs. Everything gets hashed first, so even if someone breaks into the database, they can't see any personal data.
Security Ops Dashboard	Plain HTML, CSS, JavaScript	No frameworks, no build steps, no npm install headaches. Just open it in a browser and it works. It polls the backend every 3 seconds for live updates.
DevOps & CI	Docker, GitHub Actions	Every code push automatically runs all 26 tests and builds a fresh Android APK. Docker Compose spins up the whole stack (server + database) with one command.

6 Implementation Plan

We built the system in six phases. Each phase was tested before moving to the next one:

Phase 1 — Building the Risk Engine

This was the core brain of the project. We wrote the rule-based risk calculator that takes in security events (SIM swaps, device changes, password resets) and produces a score from 0 to 100. The scoring engine looks back over a 24-hour sliding window — any event older than 24 hours drops off the score automatically. We also built the pattern detector that catches attack chains (like a SIM swap followed by a password reset from a new phone).

Phase 2 — Setting Up the Database

We designed five PostgreSQL tables: one for raw security events, one for computed risk scores, one for active alerts, one for fraud investigation cases, and one for tracking trusted devices. All tables have proper indexes so lookups by user ID and timestamp are fast. We also built an in-memory version of the same storage so the app works without a database during local testing and demos.

Phase 3 — Building the API Layer

We created 15+ REST API endpoints covering event ingestion, risk queries, alert management, fraud case workflows, health checks, and simulation controls. The event intake pipeline verifies carrier webhook signatures using HMAC-SHA256 — if the signature doesn't match, the event gets rejected. The simulation endpoints are locked behind a demo-mode flag and return HTTP 403 in production.

Phase 4 — Building the Android App

The mobile app was built entirely with programmatic Java views (no XML layouts) to keep it lightweight and self-contained. We wired up Android's native `BiometricPrompt` API for real fingerprint scanning, added high-priority push notifications for critical threats, and built the event timeline feed and simulation sandbox with 8 attack scenarios. The app has zero third-party library dependencies — just standard Android SDK APIs.

Phase 5 — Creating the Security Operations Dashboard

We built a web-based control panel that the bank's security team would use. It shows a live animated risk gauge, a filterable timeline of all security events, a case management queue for flagged transactions, and a multi-step attack replay tool for training and demos. The dashboard runs entirely in the browser with vanilla HTML/CSS/JS — no React, no Angular, no build tools.

Phase 6 — Automated Testing & CI/CD

We wrote 26 automated tests covering unit tests for the risk engine (scoring, temporal correlation, pattern detection, 24-hour window expiration), integration tests for every API endpoint, and dashboard tests for static file serving and case workflows. GitHub Actions runs these tests on every push and automatically builds the Android APK.

7 Test Results

What We Tested	What Should Happen	Result
A single SIM swap (no other suspicious activity)	Score goes to 30, user sees a warning, but account stays open	Passed
SIM swap + new device login together	Score jumps to 50, triggers "account takeover" pattern flag	Passed
Full attack chain (SIM + device + password + beneficiary + wire)	Score hits 100, transfer gets held, fraud case opens	Passed
An old SIM swap from 25 hours ago	Gets dropped from the score — it's outside the 24-hour window	Passed
User scans fingerprint and says "that was me"	Alerts get cleared, score resets to LOW, audit trail is created	Passed
Carrier webhook with a wrong signature	Event gets rejected — can't fake carrier notifications	Passed
Simulation endpoint called with demo mode off	Returns 403 Forbidden — sandbox is locked in production	Passed
Full automated suite (26 tests)	All pass with no failures	26 / 26 Passed